

PROJECT REPORT

AI Fitness & Diet Suggester

A KNN-based Personalized Workout & Diet Recommendation System

Name	Veer Jain
Registration No.	25MIM10062
Branch	Integrated M.Tech in AI
Year	2nd Year
Course Code	CSA2001
Course Title	Fundamentals in AI and ML
Submission Type	Build Your Own Project (BYOP) — Capstone Evaluation
Language / Stack	Python 3, pandas, scikit-learn (CLI application)

Table of Contents

1. Introduction	4
2. Problem Statement	4
Scope	4
Target Users	4
3. Functional Requirements	4
4. Non-Functional Requirements	5
5. System Architecture	5
6. Design Diagrams	5
6.1 Use Case Diagram	5
6.2 Process Flow / Workflow Diagram	6
6.3 Sequence Diagram	7
6.4 Component Diagram	8
6.5 Storage Schema (ER-style)	8
7. Dataset Description & Model Selection	10
Dataset	10
Model Selection Rationale	10
Evaluation Methodology	10
8. Design Decisions & Rationale	10
KNN instead of if-else rules	10
Keeping height out of the ML model	10
Showing the health snapshot before the workout plan	10
Appending to the log instead of overwriting	11
Validating input ranges, not just types	11
9. Implementation Details	11
10. Screenshots / Results	11
11. Testing Approach	12
12. Challenges Faced	13
13. Learnings & Key Takeaways	13
14. Future Enhancements	13

1. Introduction

This report documents the design, implementation, and evaluation of the **AI Fitness & Diet Suggester**, a command-line application submitted as a Build-Your-Own-Project (BYOP) capstone for CSA2001 — Fundamentals in AI and ML. The project applies a supervised machine learning technique (K-Nearest Neighbours) to a small, self-collected dataset in order to recommend a workout style, and combines that prediction with rule-based health calculations (BMI, water intake, and calorie targets) and a goal-based diet plan.

The remainder of this report covers the problem being solved, the functional and non-functional requirements, the system's architecture and design diagrams, the reasoning behind key implementation decisions, how the system was tested, and what was learned in the process.

2. Problem Statement

A useful fitness recommendation has to account for who the person actually is: their age, weight, and what they are trying to achieve. Generic advice found online or given identically to everyone ignores this, and getting continuous personal coaching is not accessible to most beginners. This project explores whether a simple, explainable machine learning model — trained on just three features — can produce workout recommendations that meaningfully differ across different users, while also surfacing standard, easy-to-compute health metrics (BMI, hydration, and calorie targets) alongside the recommendation.

Scope

- In scope: single-user CLI tool, KNN-based workout-type prediction, rule-based health metrics, goal-based diet plans, CSV-based session logging.
- Out of scope: multi-user accounts, persistent relational database, graphical interface, real-time wearable-device integration.

Target Users

Beginners starting a fitness routine without access to a personal trainer, and anyone who wants a fast, personalized starting point for workout and diet choices based on a few simple inputs.

3. Functional Requirements

- **FR1 — Personalized recommendation generation:** collect a user profile (age, weight, height, gender, goal) and produce a predicted workout type plus a matching diet plan.
- **FR2 — Health snapshot calculation:** compute and display BMI, daily water intake target, and estimated calorie needs (TDEE) from the same profile.
- **FR3 — Recommendation history management:** persist every recommendation to a CSV log, and let the user view or clear that history on demand.

- **FR4 — Input validation workflow:** every CLI prompt loops until it receives a value that is both the right type and within a realistic range.

4. Non-Functional Requirements

- **Reliability:** invalid input (wrong type, out-of-range value, blank input) never crashes the program; it re-prompts instead.
- **Usability:** every prompt states the expected format/range inline, and menu options are numbered and re-shown after every action.
- **Maintainability:** logic is split into single-responsibility modules (content, health_metrics, input_helpers, data_manager, recommender).
- **Resource efficiency / Portability:** no external services or database are required; the app runs fully offline from CSV files and a lightweight in-memory KNN model, from any working directory.
- **Error handling:** file and model errors are caught and reported with a clear message rather than a raw stack trace.

5. System Architecture

The system is organised into three layers: a thin CLI layer that owns the menu loop, an application-logic layer of five focused modules, and a storage layer of two CSV files (one for training data, one for the recommendation log). The trained KNN model sits alongside the data-manager module that owns it.

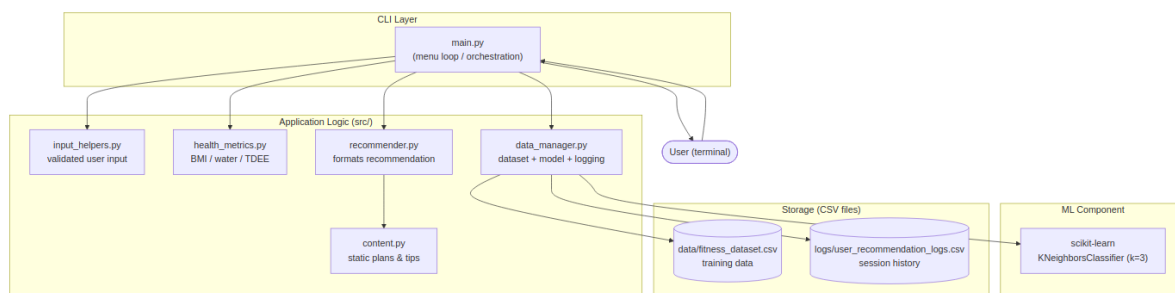


Figure 1 — System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

The user interacts with the system through four primary use cases from the main menu; the recommendation use case internally includes the health-snapshot calculation and the KNN prediction step.

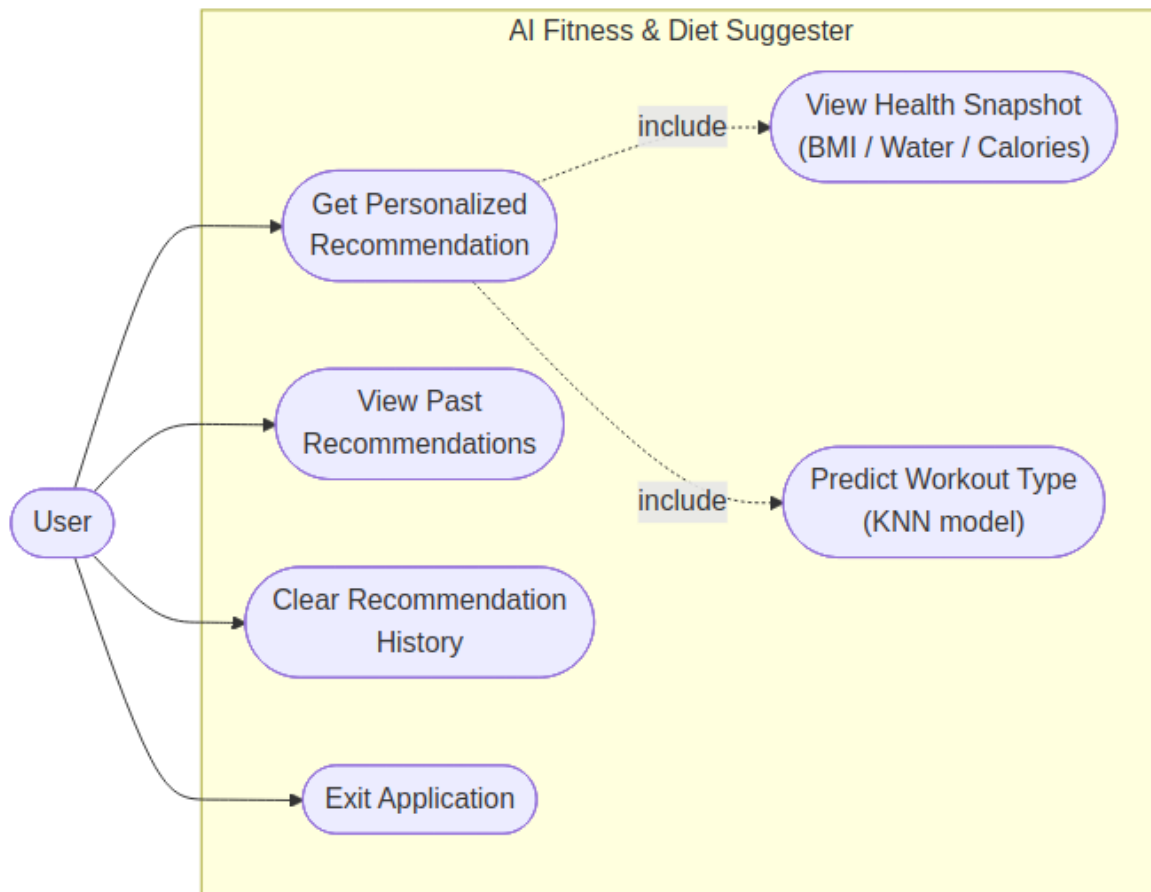


Figure 2 — Use Case Diagram

6.2 Process Flow / Workflow Diagram

This diagram traces control flow from program start, through dataset/model bootstrap, into the menu loop and each of its four branches.

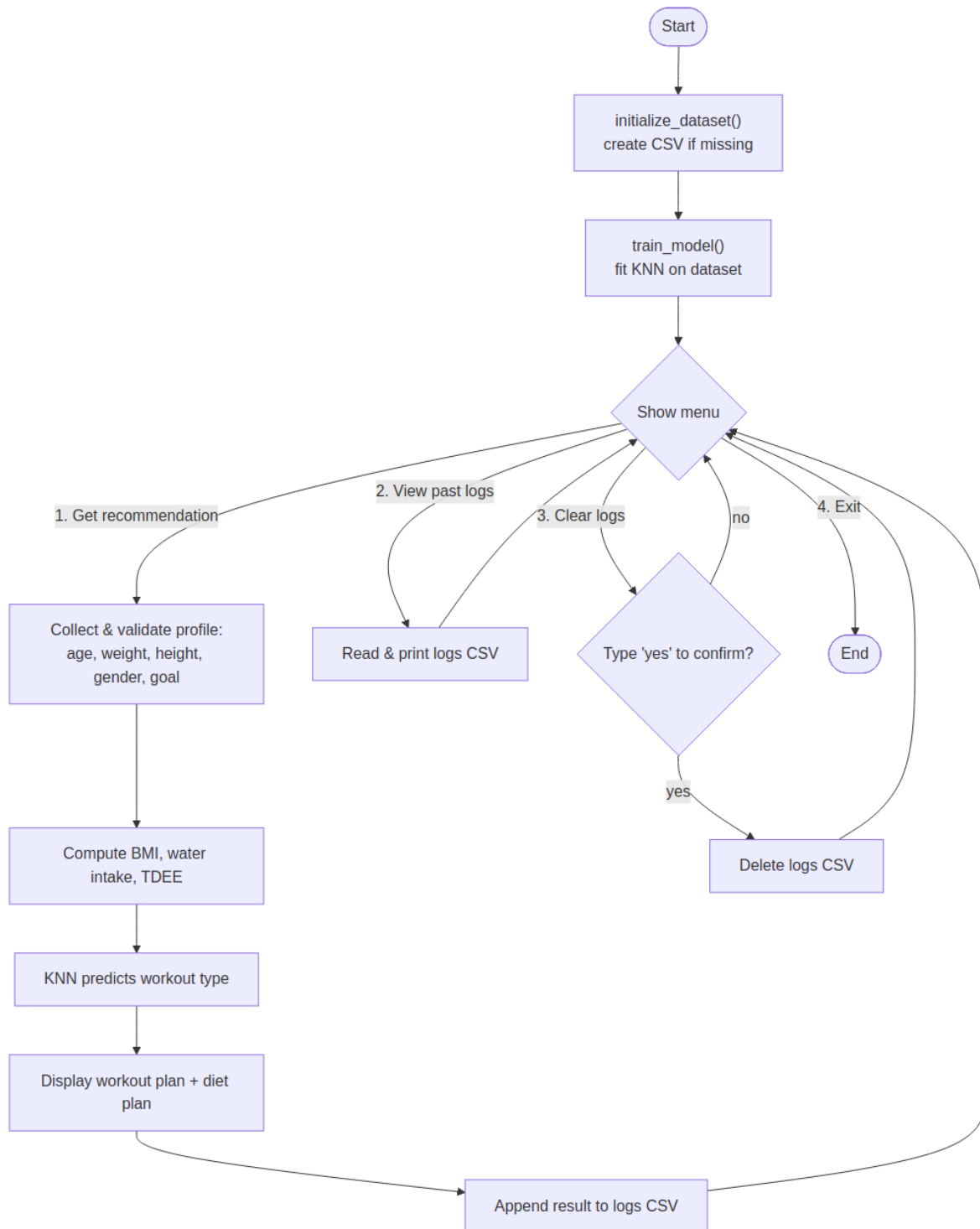


Figure 3 — Process Flow Diagram

6.3 Sequence Diagram

The sequence diagram below details the most important use case — "Get Personalized Recommendation" — showing the order of calls between the CLI, the input/health-metric helpers, the trained KNN model, and the recommendation formatter.

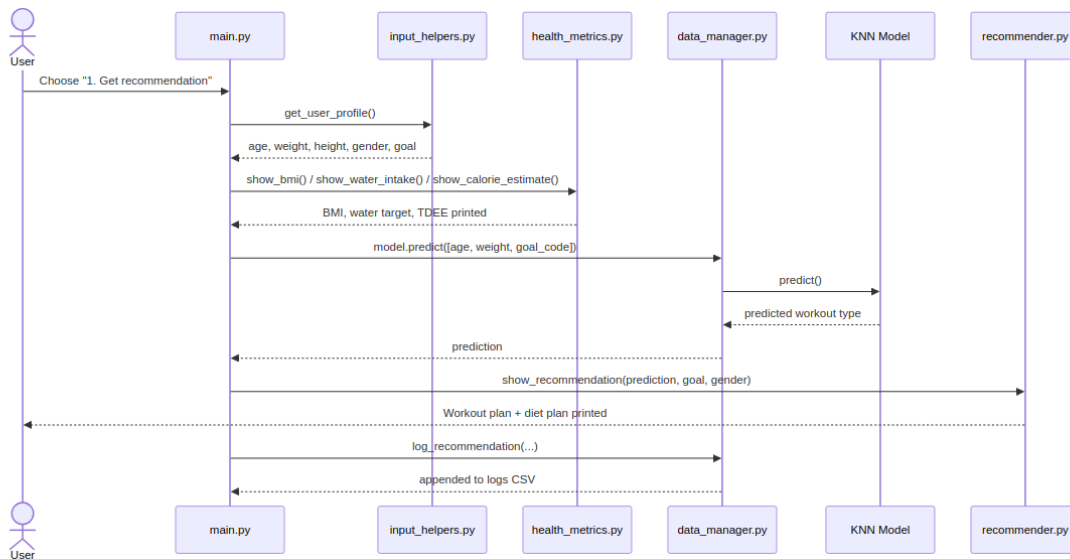


Figure 4 — Sequence Diagram (Get Recommendation)

6.4 Component Diagram

Since the implementation is function-based rather than class-based, module dependencies are shown as a component diagram instead of a UML class diagram.

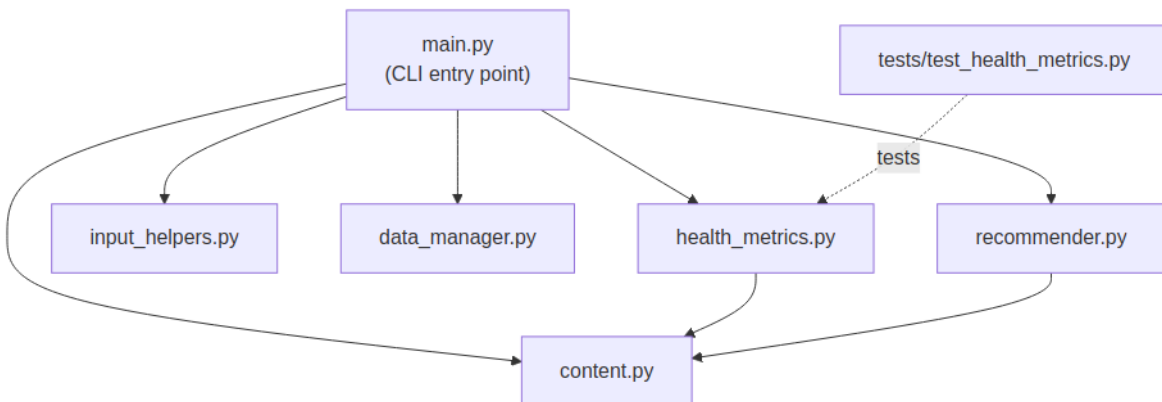


Figure 5 — Component Diagram

6.5 Storage Schema (ER-style)

There is no relational database in this project; storage is two flat CSV files. Their structure is shown below in an ER-style diagram for clarity.

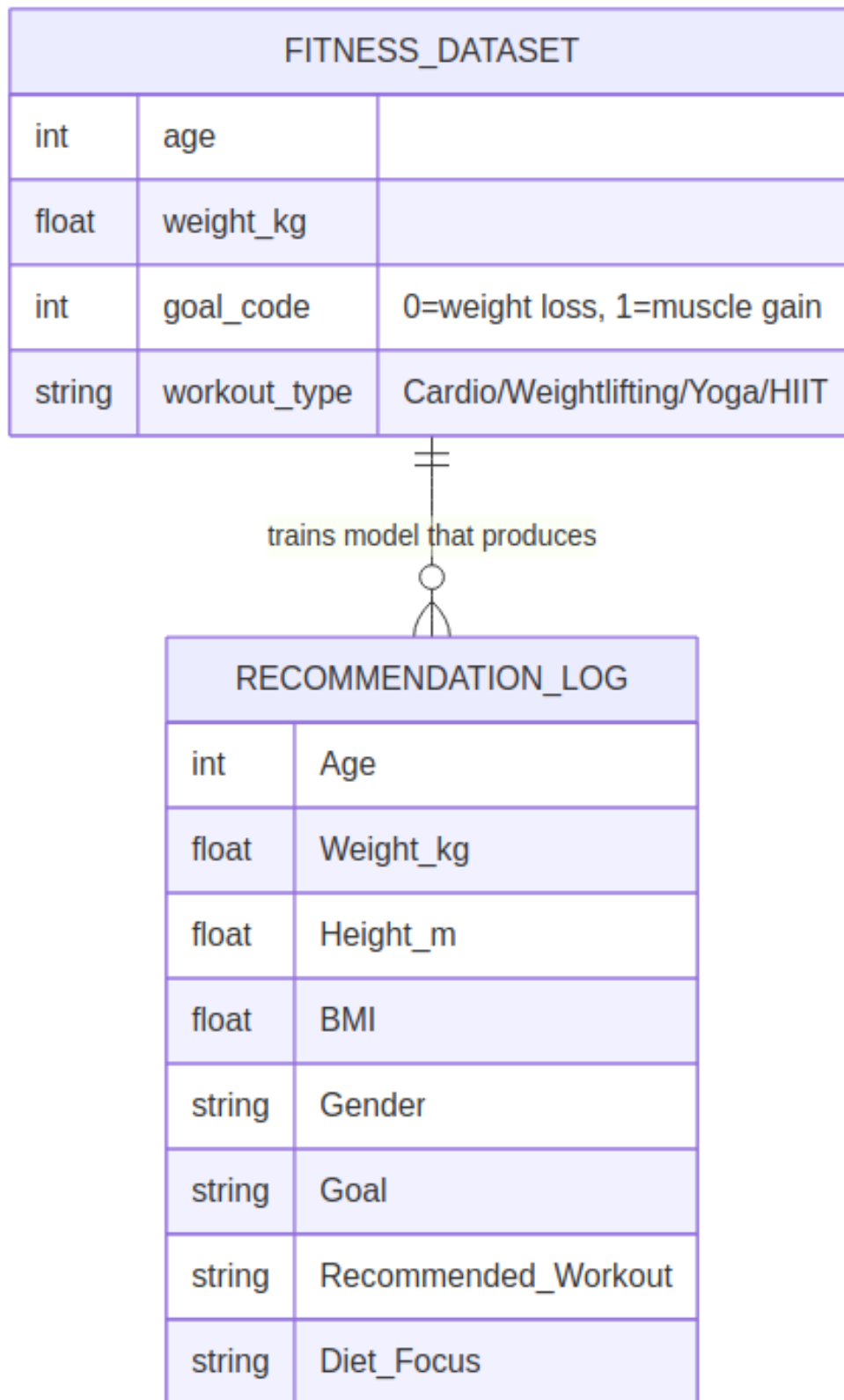


Figure 6 — Storage Schema Diagram

7. Dataset Description & Model Selection

Dataset

The training dataset (`data/fitness_dataset.csv`) has 12 hand-labelled rows with four columns: age, weight_kg, goal_code (0 = weight loss, 1 = muscle gain), and the label workout_type (Cardio, Weightlifting, Yoga, or HIIT). It is small and illustrative rather than exhaustive — enough to demonstrate the approach, not to generalise perfectly to every possible user.

Model Selection Rationale

K-Nearest Neighbours ($k=3$) was chosen because its reasoning mirrors how a human coach would informally reason: find people with a similar age, weight, and goal, and recommend what worked for them. It requires no training phase beyond loading the data, is easy to explain to a non-technical evaluator, and naturally improves as more rows are added to the dataset — unlike a hardcoded if/else rule tree, which was the first version built and quickly discarded for ignoring age entirely. Height was deliberately left out of the model's features: with only 12 training rows, adding a fourth feature diluted the neighbour search and produced inconsistent predictions for users who differed only in height. Height is still collected and used, but only for the BMI and calorie calculations where it is actually relevant.

Evaluation Methodology

Given the dataset's size (12 rows), a held-out train/test split would leave too few examples to be meaningful, so evaluation was done through manual scenario testing instead: multiple representative profiles (varied age, weight, and goal) were run through the CLI and the predicted workout type was checked for consistency with the nearest labelled rows in the dataset (see Section 10, Testing Approach). This is explicitly noted as a limitation in Section 13.

8. Design Decisions & Rationale

KNN instead of if-else rules

An early version was a chain of if/else statements (weight loss → Cardio, muscle gain → Weightlifting). It ignored age and weight entirely, so a 20-year-old and a 50-year-old with the same goal received identical advice. KNN lets those features actually influence the output.

Keeping height out of the ML model

Height is collected but passed only to the BMI/TDEE formulas, not to the KNN model (see Section 7 for the reasoning).

Showing the health snapshot before the workout plan

BMI, water intake, and calorie targets are shown before the recommendation, not after, so the user has context (e.g. their BMI category) for why a particular workout intensity is being suggested.

Appending to the log instead of overwriting

The recommendation log is opened in append mode on every run, with the header written only once, so that a user's history accumulates across sessions instead of being erased each time.

Validating input ranges, not just types

The original script only rejected non-numeric input. A real logged session had a height of 1 cm typed by mistake, silently producing a BMI of 870,000. Every prompt now also enforces a realistic real-world range and re-prompts otherwise (see Section 9 and the project README for the full list of fixes).

9. Implementation Details

The implementation is split across six files under `src/` and a `main.py` entry point, following the architecture in Section 5:

- `content.py` — static data: workout routines, diet plans, gender-specific tips, and the goal-to-code mapping.
- `health_metrics.py` — pure functions for BMI, daily water intake, and TDEE/calorie targets (Mifflin-St Jeor formula).
- `input_helpers.py` — validated CLI prompts for age, weight, height, gender, and goal, each with an enforced real-world range.
- `data_manager.py` — dataset bootstrap, KNN model training, and CSV read/append/delete logic for the recommendation log.
- `recommender.py` — formats the final workout + diet recommendation for display.
- `main.py` — the menu loop that wires the above modules together.

This is a refactor of an original single 394-line script into these six modules, along with the bug fixes summarised in Section 8 and the project README. Refer to the README's "Bug Fixes Made to the Original Script" section for the complete, itemised list.

10. Screenshots / Results

Below is a sample terminal session: a 27-year-old male, 82 kg, 180 cm, with a muscle-gain goal, requesting a recommendation and then viewing the saved log.

```
python3 main.py - terminal

=====
      AI Fitness & Diet Suggester
=====

What would you like to do?
  1.  Get a personalized fitness & diet recommendation
  2.  View my past recommendations
  3.  Clear all logs
  4.  Exit

Your choice (1, 2, 3, or 4): 1

Great! Let's learn a little about you first.

How old are you? (1-120, e.g., 21): 27
What's your current weight in kg? (20-300, e.g., 75.5): 82
What's your height in cm? (50-250, e.g., 175): 180
What's your gender? Type 'male' or 'female': male
What's your main goal? Type 'weight loss' or 'muscle gain': muscle gain

=====
      Your Health Snapshot
=====

BMI: 25.3 (Overweight)
Note: A mix of cardio and strength work will help a lot.

Daily water intake target: 2.9 liters
Tip: Spread it across the day, don't drink it all at once!

Estimated maintenance calories: ~2813 kcal/day
Caloric Surplus Target: ~3113 kcal/day
=====

Your Recommended Workout Style: Cardio
-----
Weekly Cardio & Endurance Plan:
- Monday: 30 min steady-state jogging or brisk walking.
- Tuesday: 20 min cycling or swimming.
- Thursday: 30 min interval running (1 min fast, 2 min slow).

Tip for you: Focus on maintaining a strong pace. Track your
heart rate to stay in the fat-burn zone (60-70% max HR).

Your recommendation has been saved! Keep up the great work!
```

Figure 7 — Sample CLI Session

11. Testing Approach

Two layers of testing were used:

- **Automated unit tests** (`tests/test_health_metrics.py`) cover the pure calculation functions — BMI at a known input, all four BMI category boundaries, water-intake conversion, and TDEE for both goals — and all 5 pass (verified via `python3 tests/test_health_metrics.py`).
- **Manual scenario / integration testing** of the full CLI flow: normal recommendation requests across different profiles, boundary and invalid inputs (negative age, non-numeric weight, an out-of-range height reproducing the original 870,000-BMI bug) to confirm each is now rejected and re-prompted, log viewing with and without existing history, and log clearing with both a confirmed and a declined prompt.

The ML prediction itself was checked qualitatively rather than with a held-out test set, for the reasons given in Section 7 (Evaluation Methodology).

12. Challenges Faced

- The calorie formula initially used only the male BMR constant for every user, silently overestimating female users' calorie targets by 150+ kcal, until it was checked against an independent calculator.
- Adding height as a fourth feature to the KNN model made predictions unstable across very similar users, given the dataset's small size — resolved by excluding it from the model and keeping it only for BMI/TDEE.
- Unvalidated numeric input (e.g. typing a height in the wrong unit) could silently produce nonsensical results rather than an obvious crash, which is harder to notice than a hard failure — resolved with explicit range validation.
- There is no single agreed-upon daily water-intake formula across sources; 35 ml/kg was chosen as a reasonable middle estimate after comparing several references.

13. Learnings & Key Takeaways

- Simple-looking health formulas have hidden requirements (the calorie formula needs gender; BMI needs metric units; TDEE needs an assumed activity level) that are easy to miss without reading the formula carefully.
- Real users provide input in ways the developer doesn't anticipate, which is why input validation matters more than it initially seems to.
- A 12-row dataset is enough to demonstrate that a KNN-based approach works, but not enough to trust the predictions across a wide range of real users.
- Splitting a single script into responsibility-focused modules made it noticeably easier to reason about, test, and fix (the height-validation bug touched exactly one file, `input_helpers.py`).

14. Future Enhancements

- A larger, more diverse training dataset for more trustworthy predictions across body types and age groups.
- A fitness-level input (beginner / intermediate / advanced) to adjust workout intensity, especially for HIIT.
- A graphical interface (e.g. Tkinter or a small web front-end) as an alternative to the CLI.
- Basic analysis over the accumulated recommendation logs (e.g. most frequent recommendation per user).

- More workout categories beyond the current four.

15. References

- Mifflin, M. D. et al. (1990). "A new predictive equation for resting energy expenditure in healthy individuals." *American Journal of Clinical Nutrition* — basis for the BMR/TDEE calculation.
- World Health Organization — BMI classification thresholds (Underweight / Normal / Overweight / Obese).
- scikit-learn documentation — `sklearn.neighbors.KNeighborsClassifier`.
- pandas documentation — CSV I/O (`read_csv` / `to_csv`).

— End of Report —